

Canonical and Standard Form

Boolean algebra is the foundation of digital electronics, enabling the design and analysis of logic circuits. Canonical and standard forms are key concepts in Boolean algebra, used to simplify and optimize these circuits for better performance.

- Canonical forms provide a unique representation of Boolean functions using minterms or maxterms.
- Standard forms simplify logic expressions into sum of products or product of sums for easier implementation.
- These forms help reduce hardware complexity and improve efficiency in digital design.
- Mastering canonical and standard forms is key for troubleshooting and enhancing circuit reliability.

Canonical and standard forms are crucial in Boolean algebra and digital logic design.

Canonical Forms in Boolean Algebra

Canonical forms in Boolean algebra provide a standard way to represent logic functions using minterms or maxterms.

- **Minterms:** A minterm is the product of different Boolean variables, each occurring exactly once, where the output is 1. If the value of the Boolean variable is 1, we take the variable without complementing it and if the value is 0, we take the variable with complement.
- **Maxterms:** A maxterm is the sum of different Boolean variables, each occurring exactly once, where the output is 0. If the value of the Boolean variable is 0, we take the variable without complementing it and if the value is 1, we take the variable with complement.

Truth table representing minterm and maxterm:

S. No.	X	Y	Z	Minterms (Product Terms)	Maxterms (Sum Terms)
0	0	0	0	$m_0 = X'.Y'.Z'$	$M_0 = X + Y + Z$
1	0	0	1	$m_1 = X'.Y'.Z$	$M_1 = X + Y + Z'$
2	0	1	0	$m_2 = X'.Y.Z'$	$M_2 = X + Y' + Z$
3	0	1	1	$m_3 = X'.Y.Z$	$M_3 = X + Y' + Z'$
4	1	0	0	$m_4 = X.Y'.Z'$	$M_4 = X' + Y + Z$
5	1	0	1	$m_5 = X.Y'.Z$	$M_5 = X' + Y + Z'$
6	1	1	0	$m_6 = X.Y.Z'$	$M_6 = X' + Y' + Z$
7	1	1	1	$m_7 = X.Y.Z$	$M_7 = X' + Y' + Z'$

There are two types of canonical forms that are commonly used:

1. Canonical Sum of Products (SOP)

In the Canonical Sum of Products form, every term in the Boolean expression is minterm. The sum (OR) of these minterms represents the final expression. For this sum we only consider minterms where output of Boolean function is 1. Each minterm contains all input variables either in true or in complemented form.

2. Canonical Product of Sums (POS)

In the Canonical Product of Sums form, every term in the Boolean expression is maxterm. The product (AND) of these maxterms represents the final expression. For this product we only consider maxterms where output of Boolean function is 0. Each maxterm contains all input variables either in true or in complemented form.

Example

Consider two Boolean variable A, B and a Boolean function F. The Boolean function F is equal to A XOR B.

$$F(A, B) = A \oplus B$$

Truth Table for F(A, B):

S. No.	A	B	F(A, B)	Minterms (Product Terms)	Maxterms (Sum Terms)
0	0	0	0	No minterm as output is 0	$M_0 = A + B$
1	0	1	1	$m_1 = A'.B$	No maxterm as output is 1
2	1	0	1	$m_2 = A.B'$	No maxterm as output is 1
3	1	1	0	No minterm as output is 0	$M_3 = A' + B'$

Canonical Sum of Products (SOP) Form for the Boolean Function 'F':

$$F(A, B) = m_1 + m_2 = A'.B + A.B'$$

Canonical Product of Sums (POS) Form for the Boolean Function 'F':

$$F(A, B) = M_0 + M_3 = (A + B).(A' + B')$$

These forms ensure every Boolean expression has a unique representation, making analysis and simplification easier. Every Boolean function has only one canonical SOP and one canonical POS form, ensuring consistency in digital circuit design. It is used in Karnaugh maps, Quine-McCluskey method and logic minimization to optimize circuits for speed and cost.

Difference between Canonical Form and Standard Form

In Canonical Form, every Boolean function is expressed in its most complete and precise form, based directly on its truth table. It contains every variable either in true or complemented form in its terms. The canonical forms are unique for each Boolean function because they are derived directly from the truth table without any simplification or omission of terms.

On the other hand, Standard Form offers a more flexible and simplified representation of the Boolean function. While it still involves the basic Sum of Products (SOP) or Product of Sums (POS) structure, it does not require all minterms or maxterms to be included. The Standard Forms are not unique because there can be multiple ways to simplify the function using Boolean identities, leading to different but equivalent expressions.

Feature	Canonical Form	Standard Form
Term Completeness	All variables in every term	Variables may be missing
Representation	Unique	Multiple possible forms
Complexity	Higher (more terms)	Lower (optimized terms)
Usage	Analysis, theoretical work	Practical implementation
Gate Requirement	More gates	Fewer gates
Example	$F = A'B'C + A'BC + AB'C' + ABC$	$F = A'C + AB'C' + ABC$

Conversion Between Canonical and Standard Forms

Boolean functions can be represented in canonical forms (complete minterm/maxterm expansions) or standard forms (simplified expressions). Converting between these forms is essential for circuit optimization

Converting Canonical Form into Standard Form

- Start with canonical form (SOP or POS)
- Apply Boolean algebra rules to simplify
- Eliminate redundant terms when possible

Example:

Canonical SOP: $F(A, B, C) = A'B'C + A'BC + AB'C' + ABC$

Step-by-Step Simplification:

Group terms with common variables:

$(A'B'C + A'BC) + (AB'C' + ABC)$

Factor out common terms:

$$A'C(B'+B) + AC(B' + B)$$

Apply $B'+B=1$:

$$A'C(1)+AC(1)$$

Final Standard SOP:

$$F=A'C+AC$$

Further simplification (optional):

$$F = C(A' + A) = C \text{ (Fully minimized)}$$

Standard Form: $F(A, B, C) = C$

Converting Standard Form into Canonical Form

- Identify missing variables in each product term.
- Expand using $X = X(Y + Y')$ (for SOP) or $X = X + YY'$ (for POS) to include all variables.
- Remove duplicates (if any).

Example:

$$\text{Standard SOP: } F = A + BC$$

Step-by-Step Expansion:

In first term i.e. A , B and C are missing.

$$A = A(B + B')(C + C') = ABC + AB'C + ABC' + AB'C'$$

In second term i.e. BC , A is missing.

$$BC = (A + A')BC = ABC + A'BC$$

Combining all minterms:

$$F = ABC + AB'C + ABC' + AB'C' + ABC + A'BC$$

Remove duplicates (ABC appears twice):

$$F = A'BC + AB'C' + AB'C + ABC' + ABC$$

Canonical Form: $F(A, B, C) = A'BC + AB'C' + AB'C + ABC' + ABC$

Duality Principle (Very Important)

In Boolean algebra, every valid expression remains valid if we:

- Replace + with ·
- Replace · with +
- Replace 0 with 1
- Replace 1 with 0

Types of Logic Gates

Logic gates can be broadly classified into three main categories:

AND GATE

An AND gate is used to perform logical Multiplication of binary input. The Output state of the AND gate will be high (1) if both the input is high (1), else the output state will be low(0) if any of the input is low (0).

The Boolean Expression or logic for the AND gate is the logical multiplication of inputs denoted by a full stop or single dot as :

$$X = A.B$$

The value of X will be True when both the inputs will be True.**Properties of AND Gate**

The following are two main properties of the AND gate:

1. AND gate can accept two or more than two input values at a time.
2. When all of the inputs are logic 1, the output of this gate is logic 1.

OR GATE

OR GATE is most widely used digital logic circuit. The output state of OR gate will be high i.e., (1) if any of the input state is high or 1, else output state will be low i.e., 0.

The Boolean Expression for the OR gate is the logical addition of inputs denoted by plus sign (+) as

$$X = A+B$$

The value of X will be high(true) when one of the inputs is set to high (true).

Properties of OR Gate

An OR gate have the following two properties:

1. It can have two or more input lines at a time.
2. When all of the inputs to the OR gate are low or logic 0, the output of it is low or logic 0.

NOT GATE

In digital electronics, the NOT gate is one of the basic Logic Gate having only a single input and a single output. It is also known as inverter or inverting buffer. When the input signal is "low" the output signal is "high" and vice-versa.

The Boolean expression of NOT Gate is as follows

$$Y = \bar{A} \text{ or}$$

$$Y = A'$$

the value of Y will be high when A will be low.

Properties of NOT Gate

- The output of a NOT gate is complemented or inverse of the input applied to it.
- NOT gate takes only one output.

NOR GATE

The NOR gate is the type of universal logic gate. It takes two or more inputs and gives only one output. The output state of the NOR gate will be high (1) when all the inputs are low (0). NOR gate returns the complement result of the OR gate. It is basically a combination of two basic logic gates i.e., OR gate and NOT gate.

The Boolean expression of NOR gate is as follows:

If A and B are considered as two inputs, and O as output, then the expression for a two input NOR gate will be

$$O = (A + B)'$$

The value of O will be true when all of its inputs are set to 0.

Properties of NOR Gate

The following are two important properties of NOR gate:

1. A NOR gate can have two or more inputs and gives an output.
2. A NOR gate gives a high or logic 1 output only when it's all inputs are low or logic 0.

NAND GATE

The NAND Gate is another type of Universal logic gate. The NAND gate or "Not AND" is the combination of two basic logic gates AND gate and the NOT gate connected in series. It takes two or more inputs and gives only one output. The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs the inverted operation of AND gate.

The Boolean Expression of NAND Gate is as follows

Say we have two inputs, A and B and the output is called X, then the expression is

$$X = (A \cdot B)'$$

Properties of NAND Gate

The following are the two key properties of NAND Gate

- NAND gate can take two or more inputs at a time and produces one output based on the combination of inputs applied.
- NAND gate produces a low or logic 0 output only when its all inputs are high or logic 1.

XOR GATE

In digital electronics, there is a specially designed logic gate named, XOR gate, which is used in digital circuits to perform **modulo sum**. It is also referred to as **Exclusive OR gate or Ex-OR gate**. It is used extensively in arithmetic logic circuits, logic comparators and error detection circuits. The XOR gate can take only two inputs at a time and give an output. The output of the XOR gate is high (1) only when its two inputs are dissimilar i.e., if one of them is low (0) then other one will be high (1).

Say we have two inputs, A and B and the output is called X, then the expression is

The Boolean expression of XOR Gate is as follows

$$X = A'B + AB'$$

Properties of XOR Gate

The following two are the main properties of the XOR gate:

- It can accept only two inputs at a time. There is nothing like a three or more input XOR gate.
- The output of the XOR gate is logic 1 or high, when its inputs are dissimilar.

XNOR GATE

The XNOR is the combination of XOR gate and NOT gate. The output of the XNOR gate is high(1) when both the inputs are high (1) or low(0). In other words, the output of the XNOR gate is high(1) when both the inputs are the same. the XNOR gate can sometimes be called as Equivalence gate. In simple words, The XNOR gate is the complement of the XOR gate.

The following is the Boolean expression of the XNOR gate,

$$Y = A \odot B$$

Here, A and B are the input variables and Y is the output variable.

This expression can also be written as follows,

$$Y = AB + A'B'$$

We can also express the operation of an XNOR gate using XOR gate logic as follows:

$$Y = (A \oplus B)'$$

Properties of XNOR Gate

The following are two key properties of XNOR gate:

- XNOR gate takes only two inputs and produces one output.
- The output of the XNOR gate is high or logic 1 only when it has similar inputs.

Minimization of Boolean Functions

Boolean functions are used to represent logical expressions in terms of sum of minterms or product of maxterms. Number of these literals (minterms or maxterms) increases as the complexity of the digital circuit increases. This can lead to large and inefficient circuits.

By minimizing Boolean functions, we can reduce the number of logic gates, simplify circuit design, and improve performance in terms of speed, cost, and power consumption.

For example, let the Boolean function:

$$F_2 = x'y'z + x'yz + xy'$$

This function can be further minimized by

- Grouping first two terms: $x'y'z + x'yz = x'(y'z + yz)$
- Simplify inside: $y'z + yz = z$, so it becomes $x'z$
- Add the remaining term: $x'z + xy'$

Instead of building a circuit with 3 big parts (one for each term), the minimized version needs only 2.

Various methods and techniques, such as Karnaugh maps, Quine-McCluskey algorithm, and the use of Boolean algebra, help achieve this simplification.

Main Methods for Minimizing Boolean Expressions

The two main methods for minimizing Boolean expressions are:

1. Boolean Algebra

Boolean algebra involves using a set of rules and laws (like distributive, associative, and complement laws) to simplify Boolean expressions. This method focuses on applying algebraic manipulations to reduce the complexity of the expression by eliminating redundant terms.

Law/Rule	Expression
Identity Law	$A \cdot 1 = A, A + 0 = A$
Null Law	$A \cdot 0 = 0, A + 1 = 1$
Idempotent Law	$A \cdot A = A, A + A = A$
Complement Law	$A \cdot A' = 0, A + A' = 1$
Domination Law	$A \cdot 0 = 0, A + 1 = 1$
Double Negation Law	$(A')' = A$
Distributive Law	$A \cdot (B + C) = A \cdot B + A \cdot C$

Law/Rule	Expression
De Morgan's Law	$(A \cdot B)' = A' + B'$, $(A + B)' = A' \cdot B'$
Absorption Law	$A \cdot (A + B) = A$, $A + (A \cdot B) = A$
Complementation Law	$A \cdot A' = 0$, $A + A' = 1$
Consensus Theorem	$AB + A'C + BC = AB + A'C$

Example: Simplify the Boolean function $F = AB + (AC)' + AB'C(AB + C)$.

Solution: $F = AB + (AC)' + AB'C(AB + C)$

$$= AB + A' + C' + AB'C \cdot AB + AB'C \cdot C$$

$$= AB + A' + C' + 0 + AB'C (B \cdot B' = 0 \text{ and } C \cdot C = C)$$

$$= ABC + ABC' + A' + C' + AB'C (AB = AB(C + C') = ABC + ABC')$$

$$= AC(B + B') + C'(AB + 1) + A'$$

$$= AC + C' + A' (B + B' = 1 \text{ and } AB + 1 = 1)$$

$$= AC + (AC)' = 1$$

2. K-Map

The Karnaugh Map is a graphical technique used to simplify Boolean expressions by grouping adjacent cells containing 1s (minterms). This visual method makes it easier to identify patterns and minimize the expression by combining terms that can be grouped together. It is especially useful for functions with 4 or fewer variables.

Questions Based on Minimization of Boolean Functions

Example: Minimize the following Boolean function using algebraic manipulation

$$F = ABC'D' + ABC'D + AB'C'D + ABCD + AB'CD + ABCD' + AB'CD'$$

Solution:

1. Using Boolean Laws

$$\begin{aligned}
 F &= ABC'(D' + D) + AB'C'D + ACD(B + B') + ACD'(B + B') \\
 &= ABC' + AB'C'D + ACD + ACD' \\
 &= ABC' + AB'C'D + AC(D + D') \\
 &= ABC' + AB'C'D + AC \\
 &= A(BC' + C) + AB'C'D \\
 &= A(B + C) + AB'C'D
 \end{aligned}$$

$$\begin{aligned}
&= AB + AC + AB'C'D \\
&= AB + AC + AC'D \\
&= AB + AC + AD
\end{aligned}$$

Karnaugh Map or K-Map

KARNAUGH MAP OR K-MAP

Boolean functions using Boolean postulates and theorems. It is a time consuming process and re-write the simplified expressions after each step.

To overcome this difficulty, Karnaugh introduced a method for simplification of Boolean functions in an easy way. This method is known as Karnaugh map method or K-map method. It is a graphical method, which consists of 2^n cells for 'n' variables. The adjacent cells are differed only in single bit position.

K-Map method is most suitable for minimizing Boolean functions of 2 variables to 5 variables.

2 Variable K-Map

The number of cells in 2 variable K-map is four (2^2) since the number of variables is two. The following figure shows 2 variable K-Map.

		Z	0	1
Y	0	m_0	m_1	
1	m_2	m_3		

- There is only one possibility of grouping 4 adjacent min terms.
- The possible combinations of grouping 2 adjacent min terms are $\{(m_0, m_1), (m_2, m_3), (m_0, m_2) \text{ and } (m_1, m_3)\}$.

3 Variable K-Map

The number of cells in 3 variable K-map is eight (2³), since the number of variables is three. The following figure shows 3 variable K-Map.

YZ \ X	00	01	11	10
0	m ₀	m ₁	m ₃	m ₂
1	m ₄	m ₅	m ₇	m ₆

- There is only one possibility of grouping 8 adjacent min terms.
- The possible combinations of grouping 4 adjacent min terms are {(m₀, m₁, m₃, m₂), (m₄, m₅, m₇, m₆), (m₀, m₁, m₄, m₅), (m₁, m₃, m₅, m₇), (m₃, m₂, m₇, m₆) and (m₂, m₀, m₆, m₄)}.
- The possible combinations of grouping 2 adjacent min terms are {(m₀, m₁), (m₁, m₃), (m₃, m₂), (m₂, m₀), (m₄, m₅), (m₅, m₇), (m₇, m₆), (m₆, m₄), (m₀, m₄), (m₁, m₅), (m₃, m₇) and (m₂, m₆)}.
- If x=0, then 3 variable K-map becomes 2 variable K-map.

4 Variable K-Map

The number of cells in 4 variable K-map is sixteen (2⁴), since the number of variables is four. The following figure shows 4 variable K-Map.

YZ \ WX	00	01	11	10
00	m ₀	m ₁	m ₃	m ₂
01	m ₄	m ₅	m ₇	m ₆
11	m ₁₂	m ₁₃	m ₁₅	m ₁₄
10	m ₈	m ₉	m ₁₁	m ₁₀

- There is only one possibility of grouping 16 adjacent min terms.

- Let R_1, R_2, R_3 , and R_4 , represents the min terms of first row, second row, third row and fourth row respectively. Similarly, C_1, C_2, C_3 and C_4 represents the min terms of first column, second column, third column and fourth column respectively. The possible combinations of grouping 8 adjacent min terms are $\{(R_1, R_2), (R_2, R_3), (R_3, R_4), (R_4, R_1), (C_1, C_2), (C_2, C_3), (C_3, C_4), (C_4, C_1)\}$. If $w=0$, then 4 variable K-map becomes 3 variable K-map.

5 Variable K-Map

The number of cells in 5 variable K-map is thirty-two (32), since the number of variables is 5. The following figure shows 5 variable K-Map.

$v=0$

		YZ			
		00	01	11	10
WX	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

$v=1$

		YZ			
		00	01	11	10
WX	00	m_{16}	m_{17}	m_{19}	m_{18}
	01	m_{20}	m_{21}	m_{23}	m_{22}
	11	m_{28}	m_{29}	m_{31}	m_{30}
	10	m_{24}	m_{25}	m_{27}	m_{26}

- There is only one possibility of grouping 32 adjacent min terms.
- There are two possibilities of grouping 16 adjacent min terms. i.e., grouping of min terms from m_0 , to m_{15} , and m_{16} to m_{31} .
- If $v=0$, then 5 variable K-map becomes 4 variable K-map.

Minimization of Boolean Functions using K-Maps

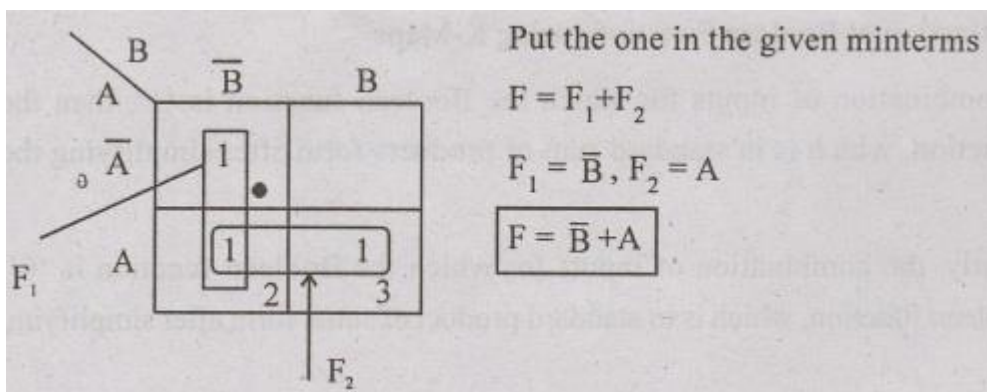
The combination of inputs for which the Boolean function is '1', then the Boolean function, which is in standard sum of products form after simplifying the K-map.

Similarly, the combination of inputs for which the Boolean function is '0', then the Boolean function, which is in standard product of sums form after simplifying the K-map.

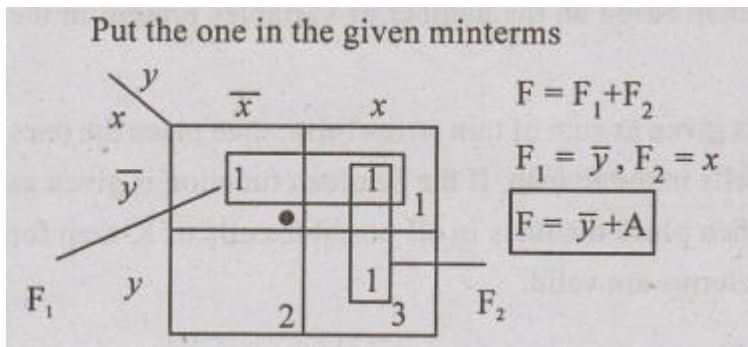
Follow these rules for simplifying K-maps in order to get standard sum of products form.

- Select the respective K-map based on the number of variables present in the Boolean function.
- If the Boolean function is given as sum of min terms form, then place the ones at respective min term cells in the K-map. If the Boolean function is given as sum of products form, then place the ones in all possible cells of K-map for which the given product terms are valid.
- Check for the possibilities of grouping maximum number of adjacent ones. It should be powers of two. Start from highest power of two and upto least power of two. Highest power is equal to the number of variables considered in K-map and least power is zero.
- Each grouping will give either a literal or one product term. It is known as prime implicant. The prime implicant is said to be essential prime implicant, if at least single '1' is not covered with any other groupings but only that grouping covers.
- Note down' all the prime implicants and essential prime implicants. The simplified Boolean function contains all essential prime implicants and only the required prime implicants.

Reduce the function using K-map $F(A,b) = \Sigma(0,2,3)$



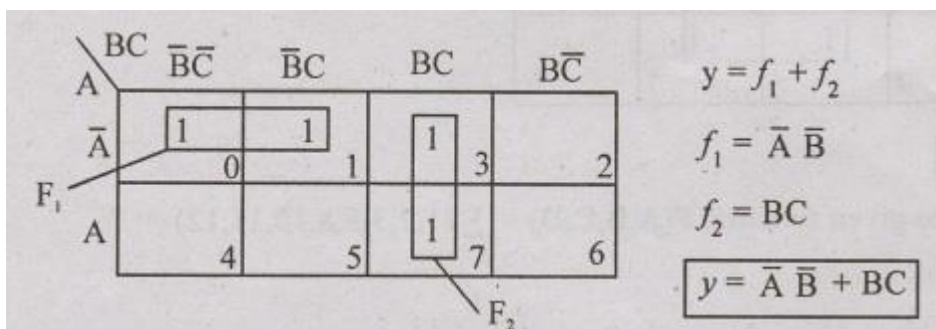
Simplify the function using Karnaugh Map $F(x,y) = \Sigma(0,1,3)$



Reduce the function $y(A,B,C) = \Sigma (0,1,3,6,7)$

Solution

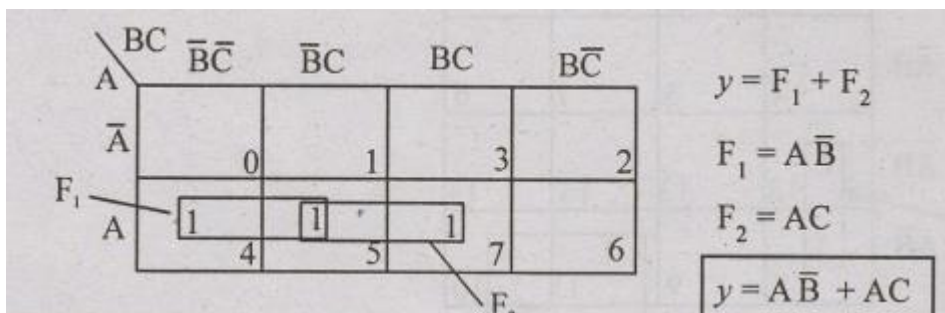
Here the 3 variable as in the function so we draw 3 variable k-map and fill ones in the given minterms



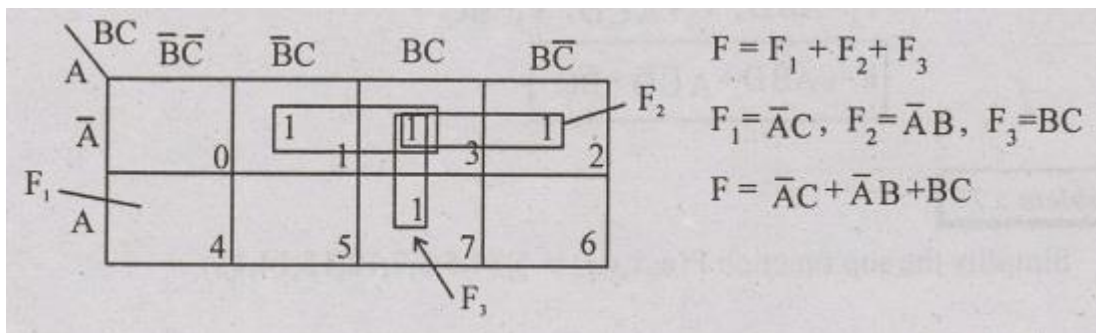
Reduce the given function $y = \Sigma (4,5,7)$

Solution:

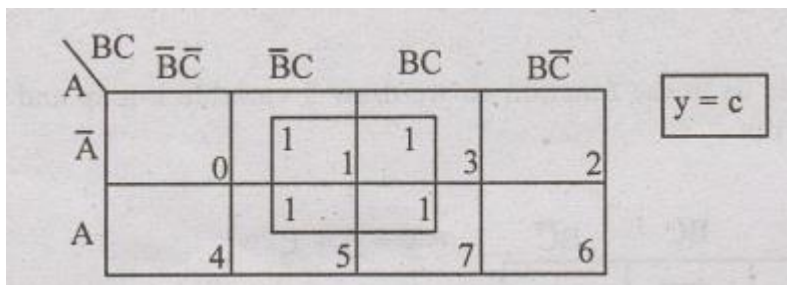
Here the 3 variable we use 3 variable k-map and put ones in the k-map



Simplify the sop function $F(A,B,C) = \Sigma (1,2,3,7)$



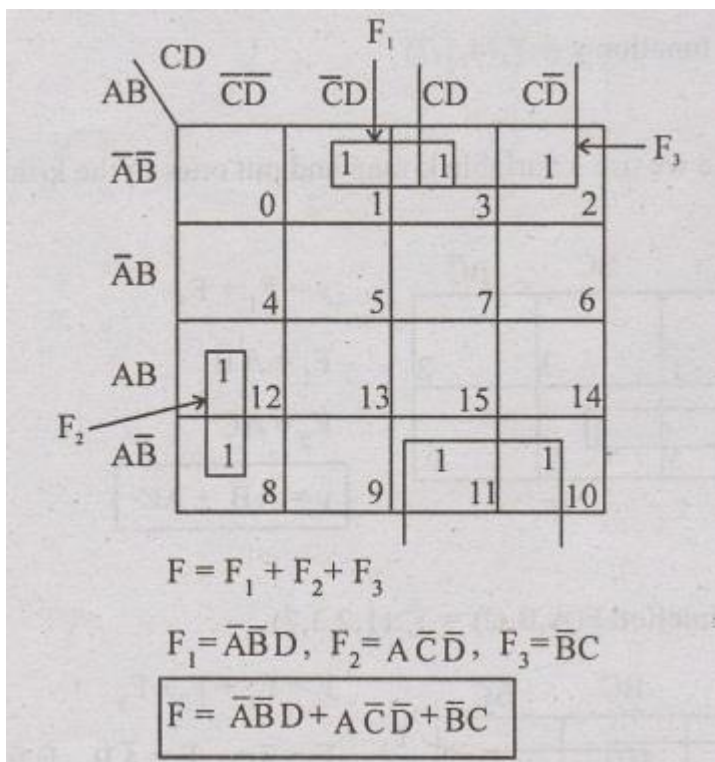
Simply the sop function $F(A,B,C) = \Sigma (1,3,5,7)$



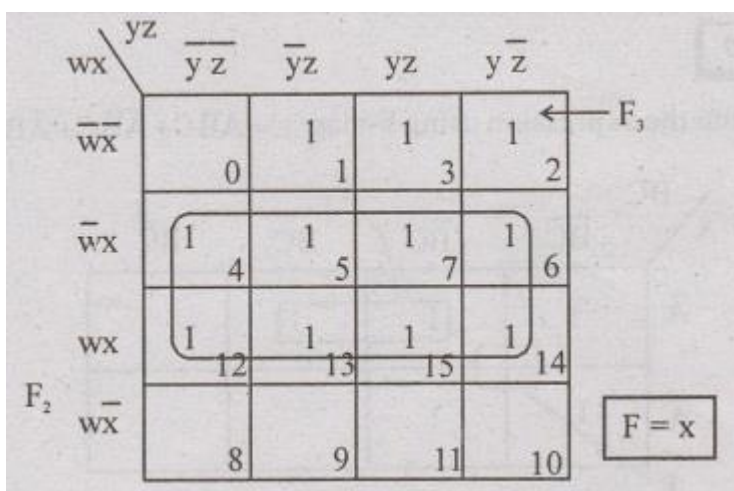
Reduce the given function $F(A,B,C,D) = \Sigma (1,2,3,5,8,10,11,12)$

Solution:

In this problem 4 variable is given use 4 variable k-map



Simplify the sop function $F(w,x,y,z) = \Sigma (4,5,6,7,12,13,14,15)$



Simplify the sop function $F(A,B,C) = \pi (0,1,6,7)$

Solution:

We use 3 variable and given pos put zero in given terms

A \ BC	$\overline{B}\overline{C}$	$\overline{B}C$	BC	$B\overline{C}$
	0	1	3	2
$\overline{A}$	0			
A			0	0

$$\overline{F} = \overline{A}\overline{B} + \overline{A}B$$

$$\overline{\overline{F}} = \overline{\overline{A}\overline{B} + \overline{A}B}$$

$$F = (\overline{\overline{A}\overline{B}}) + (\overline{\overline{A}B})$$

$$F = (\overline{\overline{A}}\overline{\overline{B}}) + \overline{\overline{A}}\overline{B}$$

$$F = (A+B) + (A+\overline{B})$$

Simplify the sop function $F(x,y,z) = \pi (0,1,2,3,5,7)$

Solution:

We use 3 variable k-map and put zero in the given maxterms

x \ yz	$\overline{y}\overline{z}$	$\overline{y}z$	$y\overline{z}$	yz
	0	1	3	2
$\overline{x}$	0	0	0	0
x		0	0	

$$\overline{F} = \overline{x} + z$$

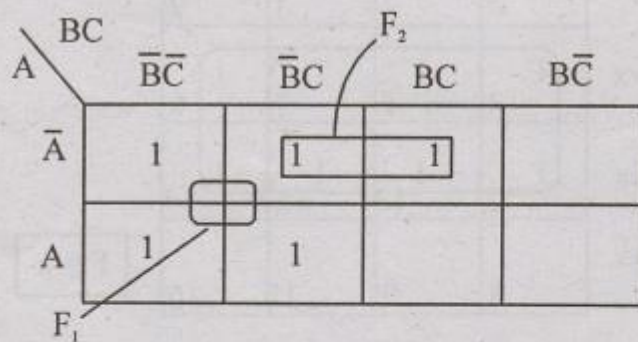
$$\overline{\overline{F}} = \overline{\overline{x} + z}$$

$$F = \overline{\overline{x}} \cdot \overline{z}$$

$$F = x\overline{z}$$

Minimum the expression using k-map $y = ABC + \overline{A}BC + AB\overline{C} + \overline{A}B\overline{C} + ABC + \overline{A}BC$

Minimum the expression using k-map $y = A\bar{B}C + \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}$



$$y = F_1 + F_2$$

$$F_1 = \bar{B}$$

$$F_2 = \bar{A}C$$

$$y = \bar{B} + \bar{A}C$$

Reduce the following function using k-map and implement in basic gates f(A, B,C)
 $AC + ABC + BC$

Solution :

The given SOP is not in standard SOP form so convert them into standard SOP

$$f(A, B, C) = \bar{A}C + A\bar{B}C + \bar{B}\bar{C}$$

Solution :

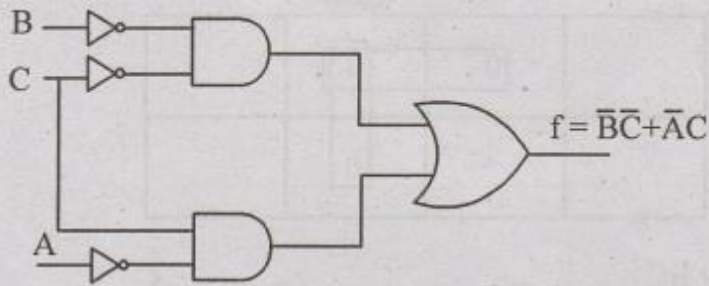
The given SOP is not in standard SOP form so convert them into standard SOP

$$f(A, B, C) = \bar{A}(B + \bar{B})C + (A + \bar{A})\bar{B}\bar{C}$$

$$f(A, B, C) = \bar{A}BC + \bar{A}\bar{B}C + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}$$

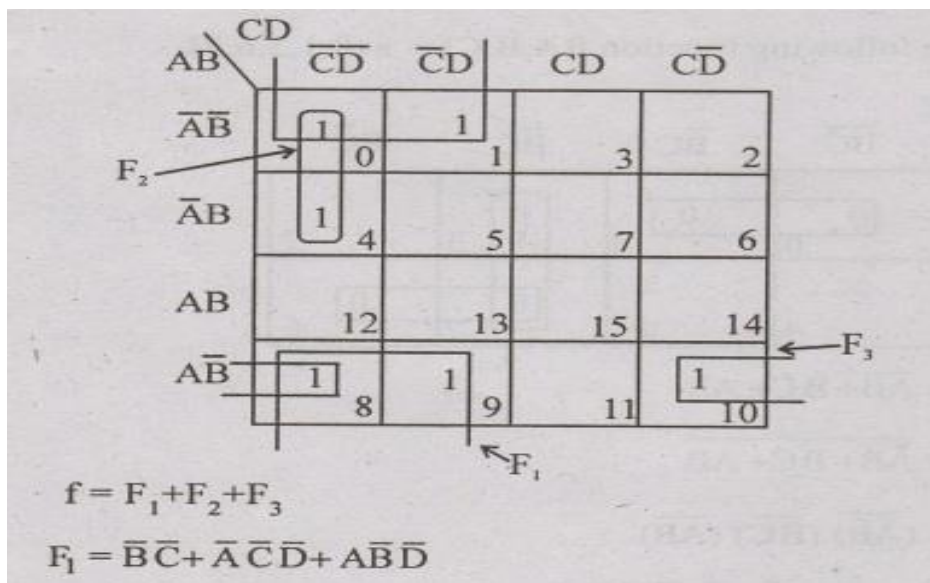
A \ BC				
	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	1	1	1	0
A	1	0	0	0

$$f = \bar{B}\bar{C} + \bar{A}C$$

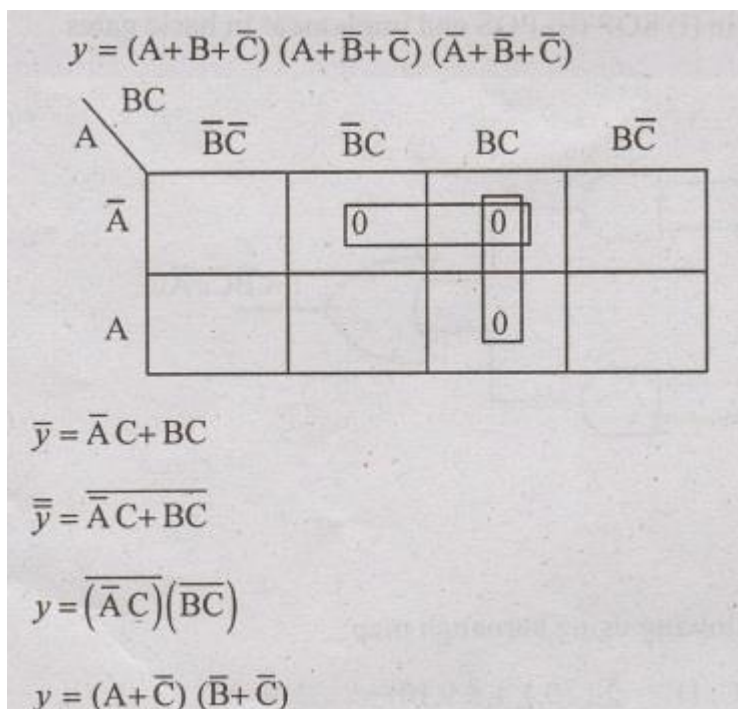


Reduce the following using karnaugh map $f(A, B, C, D) = \sum_m (0,1,4,8,9,10)$

Solution:



Minimum the expression



Minimize the following function $f(A,B,C) = \pi (0,1,3,6,7)$

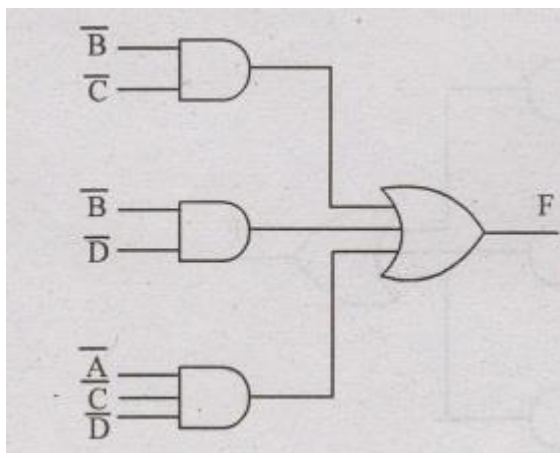
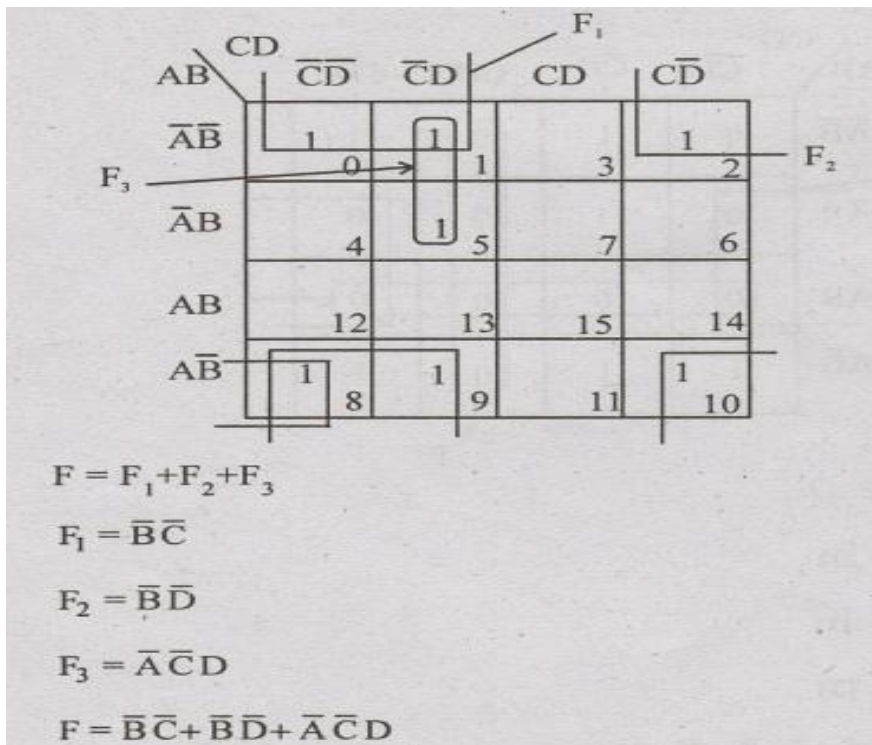
A \ BC				
	$\overline{B}\overline{C}$	$\overline{B}C$	BC	$B\overline{C}$
$\overline{A}$	0	1	3	2
A	4	5	7	6

$\overline{f} = \overline{A}\overline{B} + \overline{B}C + AB$
 $\overline{\overline{f}} = \overline{\overline{A}\overline{B} + \overline{B}C + AB}$
 $f = (\overline{\overline{A}\overline{B}})(\overline{\overline{B}C})(\overline{AB})$
 $f = (A+B)(\overline{B}+\overline{C})(\overline{A}+\overline{B})$

Simplify the following in (i) SOP (ii) POS and implement in basic gates $F(A, B, C, D) = \Sigma(0,1,2,5,8,9,10)$

Solution:

(i) SOP put one's in the given numbers



(ii) Product of sum (POS) zero's

$\Sigma_m(0,1,2,5,8,9,10) \pi(3,4,6,7,11,12,13,14,15)$

		CD			
AB		$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$		1	1	0	1
$\bar{A}B$	F_3	0	1	0	0
$A\bar{B}$		0	0	0	0
AB		0	0	0	0
$A\bar{B}$		1	1	0	1

F_1

F_2

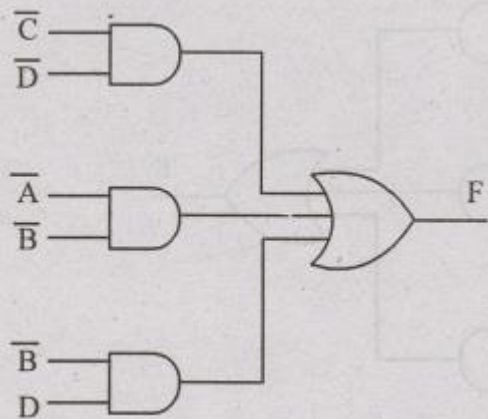
$$F = F_1 + F_2 + F_3$$

$$F_1 = (\bar{C} + \bar{D})$$

$$F_2 = (\bar{A} + \bar{B})$$

$$F_3 = (\bar{B} + D)$$

$$F = (\bar{C} + \bar{D})(\bar{A} + \bar{B})(\bar{B} + D)$$



COMBINATIONAL CIRCUIT DESIGN- Encoder and Decoder

DECODERS:

A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines. The general structure of decoder circuit is

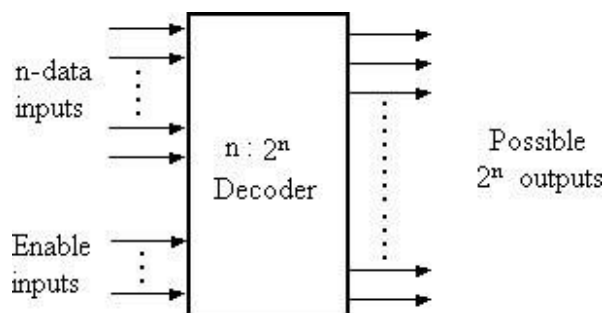


Fig : 2.24 - General structure of decoder

The encoded information is presented as n inputs producing 2^n possible outputs. The 2^n output values are from 0 through 2^n-1 . A decoder is provided with enable inputs to activate decoded output based on data inputs. When any one enable input is unasserted, all outputs of decoder are disabled.

Binary Decoder (2 to 4 decoder):

A binary decoder has n bit binary input and a one activated output out of 2^n outputs. A binary decoder is used when it is necessary to activate exactly one of 2^n outputs based on an n -bit input value.

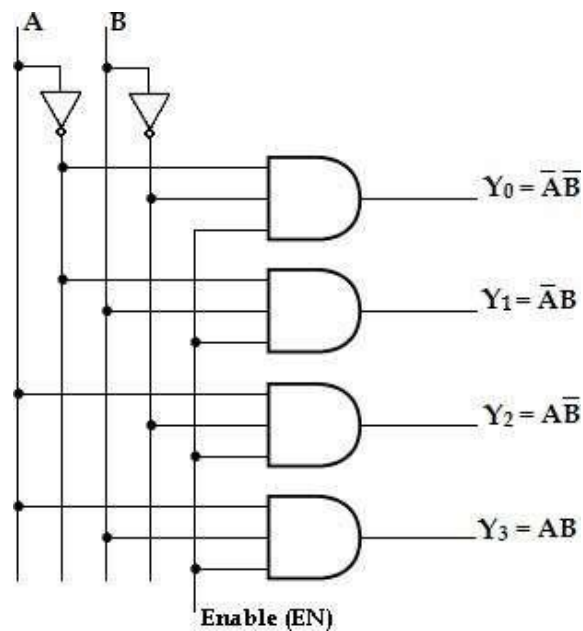


Fig : 2.25 - 2-to-4 Line decoder

Here the 2 inputs are decoded into 4 outputs, each output representing one of the minterms of the two input variables.

Inputs			Outputs			
Enable	A	B	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

As shown in the truth table, if enable input is 1 (EN= 1) only one of the outputs ($Y_0 - Y_3$), is active for a given input.

The output Y_0 is active, ie., $Y_0 = 1$ when inputs $A = B = 0$,

Y_1 is active when inputs, $A=0$ and $B=1$, Y_2

is active, when input $A=1$ and $B=0$, Y_3 is

active, when inputs $A=B=1$.

3-to-8 Line Decoder:

A 3-to-8 line decoder has three inputs (A, B, C) and eight outputs ($Y_0 - Y_7$). Based on the 3 inputs one of the eight outputs is selected.

The three inputs are decoded into eight outputs, each output representing one of the minterms of the 3-input variables. This decoder is used for binary-to-octal conversion. The input variables may represent a binary number and the outputs will represent the eight digits in the octal number system. The output variables are mutually exclusive because only one output can be equal to 1 at any one time. The output line whose value is equal to 1 represents the minterm equivalent of the binary number presently available in the input lines.

Inputs			Outputs							
A	B	C	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

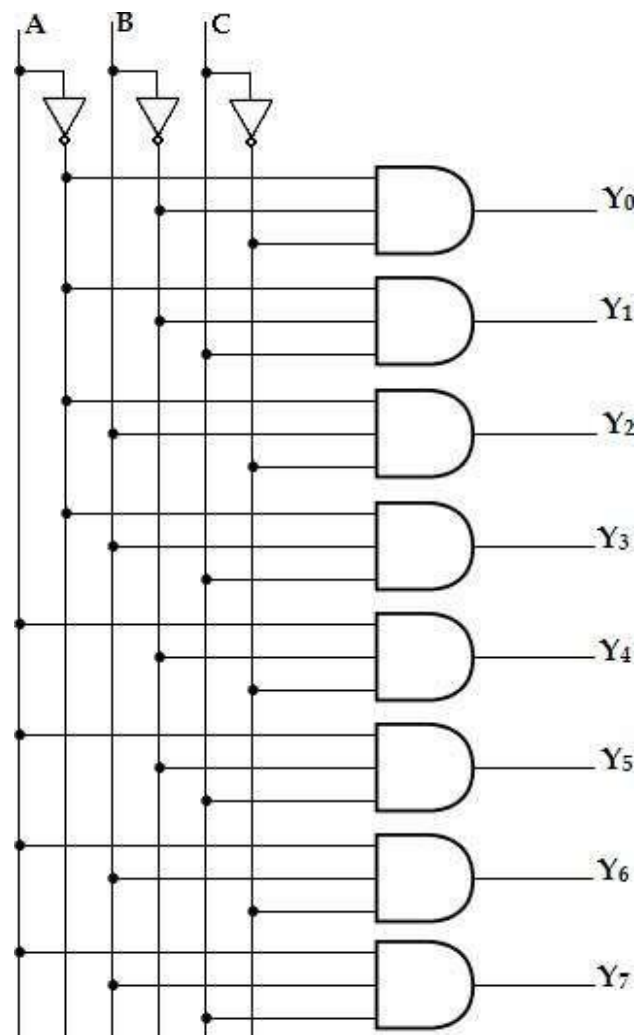
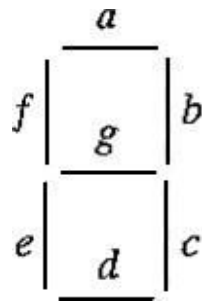


Fig : 2.26 - 3-to-8 line decoder

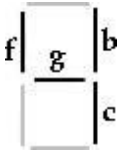
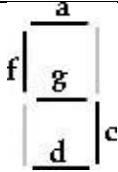
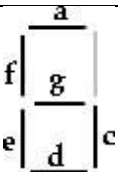
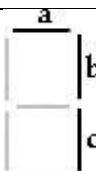
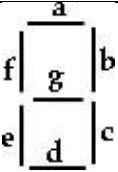
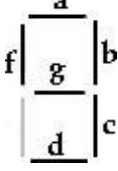
BCD to 7-Segment Display Decoder:

A seven-segment display is normally used for displaying any one of the decimal digits, 0 through 9. A BCD-to-seven segment decoder accepts a decimal digit in BCD and generates the corresponding seven-segment code.



Each segment is made up of a material that emits light when current is passed through it. The segments activated during each digit display are tabulated as—

Digit	Display	Segments Activated
0		a, b, c, d, e, f
1		b, c
2		a, b, d, e, g
3		a, b, c, d, g

4		b, c, f, g
5		a, c, d, f, g
6		a, c, d, e, f, g
7		a, b, c
8		a, b, c, d, e, f, g
9		a, b, c, d, f, g

Truth table:

	BCD code				7-Segment code						
Digit	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0

2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

K-map Simplification:

For (a)

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

$$a = A + C + BD + B'D'$$

For (b)

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	X	X	X	X
10	1	1	X	X

$$b = B' + C'D' + CD$$

For (c)

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	X	X	X	X
10	1	1	X	X

$$c = B + C' + D$$

For (d)

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	X	X	X	X
10	1	1	X	X

$$d = B'D' + CD' + BC'D + B'C + A$$

		For (e)						For (f)			
AB	CD	00	01	11	10	AB	CD	00	01	11	10
	00	1	0	0	1		00	1	0	0	0
	01	0	0	0	1		01	1	1	0	1
	11	X	X	X	X		11	X	X	X	X
	10	1	0	X	X		10	1	1	X	X
$e = B'D' + CD'$						$f = A + C'D' + BC' + BD'$					
		For (g)									
AB	CD	00	01	11	10	AB	CD	00	01	11	10
	00	0	0	1	1		00	0	0	1	1
	01	1	1	0	1		01	1	1	0	1
	11	X	X	X	X		11	X	X	X	X
	10	1	1	X	X		10	1	1	X	X
$g = A + BC' + B'C + CD'$											

Applications of decoders:

1. Decoders are used in counter system.
2. They are used in analog to digital converter.
3. Decoder outputs can be used to drive a display system.

ENCODERS:

An encoder is a digital circuit that performs the inverse operation of a decoder. Hence, the opposite of the decoding process is called encoding. An encoder is a combinational circuit that converts binary information from 2^n input lines to a maximum of n unique output lines. The general structure of encoder circuit is –

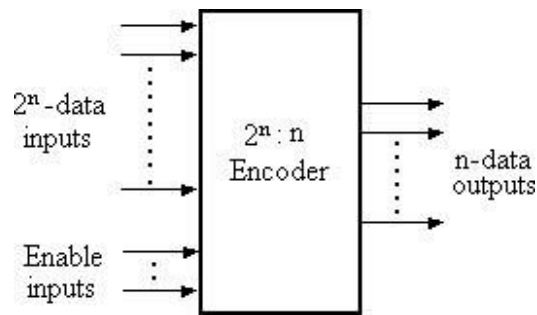


Fig : 2.27 - General structure of Encoder

It has 2^n input lines, only one which 1 is active at any time and n output lines. It encodes one of the active inputs to a coded binary output with n bits. In an encoder, the number of outputs is less than the number of inputs.

Octal-to-Binary Encoder:

It has eight inputs (one for each of the octal digits) and the three outputs that generate the corresponding binary number. It is assumed that only one input has a value of 1 at any given time.

Inputs								Outputs		
D0	D1	D2	D3	D4	D5	D6	D7	A	B	C
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

The encoder can be implemented with OR gates whose inputs are determined directly from the truth table. Output z is equal to 1, when the input octal digit is 1 or 3 or 5 or 7. Output y is 1 for octal digits 2, 3, 6, or 7 and the output is 1 for digits 4, 5, 6 or 7. These conditions can be expressed by the following output Boolean functions:

$$Z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

The encoder can be implemented with three OR gates. The encoder defined in the below table, has the limitation that only one input can be active at any given time. If two inputs are active simultaneously, the output produces an undefined combination.

For eg., if D_3 and D_6 are 1 simultaneously, the output of the encoder may be 111. This does not represent either D_6 or D_3 . To resolve this problem, encoder circuits must establish an input priority to ensure that only one input is encoded. If we establish a higher priority for inputs with higher subscript numbers and if D_3 and D_6 are 1 at the same time, the output will be 110 because D_6 has higher priority than D_3 .

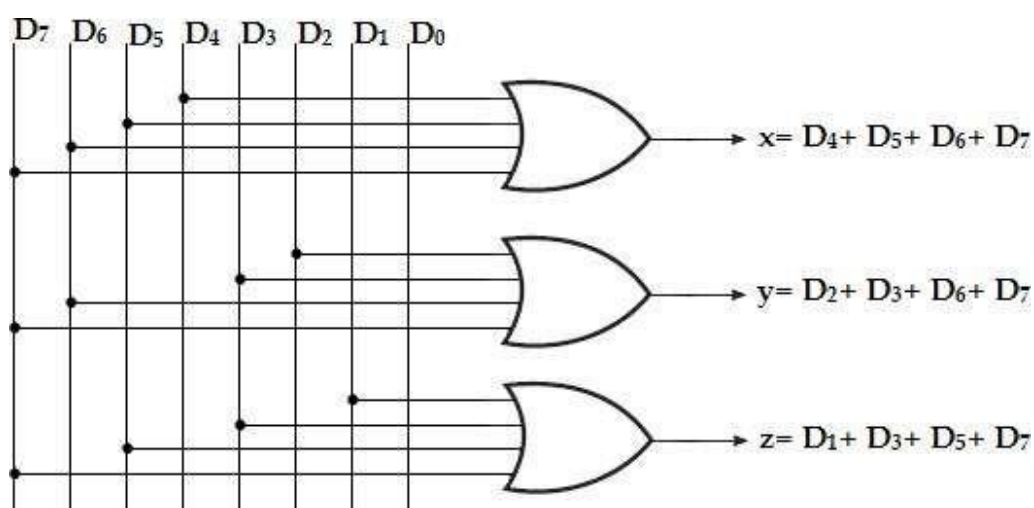


Fig : 2.28 - Octal-to-Binary Encoder

Another problem in the octal-to-binary encoder is that an output with all 0's is generated when all the inputs are 0; this output is same as when D_0 is equal to 1. The discrepancy can be resolved by providing one more output to indicate that atleast one input is equal to 1.

Priority Encoder:

A priority encoder is an encoder circuit that includes the priority function. In priority encoder, if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.

In addition to the two outputs x and y , the circuit has a third output, V (valid bit indicator). It is set to 1 when one or more inputs are equal to 1. If all inputs are 0, there is no valid input and V is equal to 0.

The higher the subscript number, higher the priority of the input. Input D_3 , has the highest priority. So, regardless of the values of the other inputs, when D_3 is 1, the output for xy is 11.

D_2 has the next priority level. The output is 10, if $D_2 = 1$ provided $D_3 = 0$. The output for D_1 is generated only if higher priority inputs are 0, and so on down the priority levels.

Truth table:

Inputs				Outputs		
D0	D1	D2	D3	x	y	V
0	0	0	0	x	x	0
1	0	0	0	0	0	1
x	1	0	0	0	1	1
x	x	1	0	1	0	1

x	x	x	1	1	1	1
---	---	---	---	---	---	---

Although the above table has only five rows, when each don't care condition is replaced first by 0 and then by 1, we obtain all 16 possible input combinations. For example, the third row in the table with X100 represents minterms 0100 and 1100. The don't care condition is replaced by 0 and 1 as shown in the table below.

Modified Truth table:

Inputs				Outputs		
D0	D1	D2	D3	x	y	V
0	0	0	0	x	x	0
1	0	0	0	0	0	1
0	1	0	0	0	1	1
1	1	0	0			
0	0	1	0	1	0	1
0	1	1	0			
1	0	1	0			
1	1	1	0			
0	0	0	1	1	1	1
0	0	1	1			
0	1	0	1			
0	1	1	1			
1	0	0	1			
1	0	1	1			
1	1	0	1			
1	1	1	1			

K-map Simplification:

$D_0D_1 \backslash D_2D_3$		<u>For X</u>			
		00	01	11	10
00		x	1	1	1
01		0	1	1	1
11		0	1	1	1
10		0	1	1	1

$x = D_2 + D_3$

$D_0D_1 \backslash D_2D_3$		<u>For y</u>			
		00	01	11	10
00		x	1	1	0
01		1	1	1	0
11		1	1	1	0
10		0	1	1	0

$y = D_3 + D_1D_2$

$D_0D_1 \backslash D_2D_3$		<u>For V</u>			
		00	01	11	10
00		0	1	1	1
01		1	1	1	1
11		1	1	1	1
10		1	1	1	1

$V = D_0 + D_1 + D_2 + D_3$

The priority encoder is implemented according to the above Boolean functions.

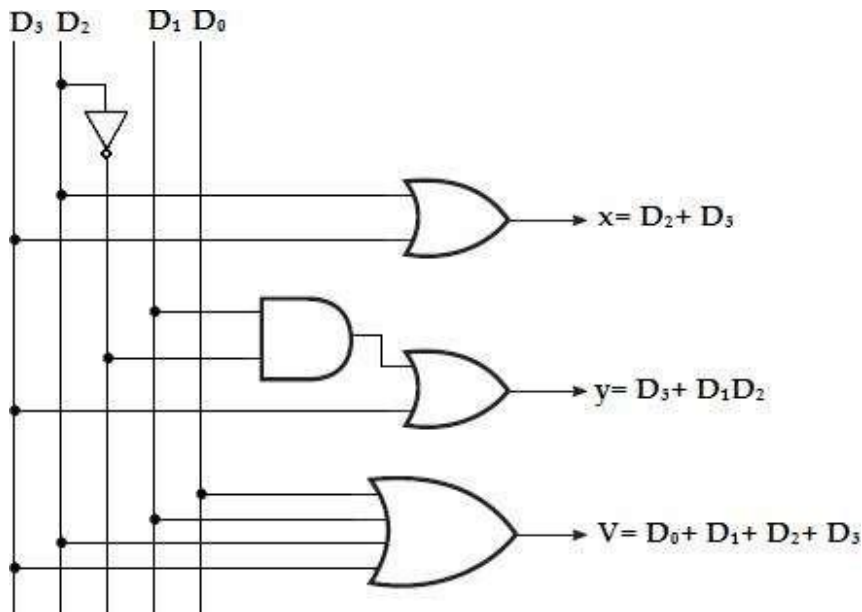


Fig : 2.29 - 4-Input Priority Encoder

Multiplexers

A multiplexer is a combinational circuit that has many data inputs and a single output, depending on control or select inputs. For N input lines, $\log_2(N)$ selection lines are required, or equivalently, for 2^n input lines, n selection lines are needed.

- Multiplexers are also known as "N-to-1 selectors," parallel-to-serial converters, many-to-one circuits, and universal logic circuits.
- They are mainly used to increase the amount of data that can be sent over a network within a certain amount of time and bandwidth.

Multiplexer

Types of Mux

The Mux can be of different types based on input but in this article, we will go through two major types of mux, which are

- **2x1 Mux**
- **4x1 Mux**

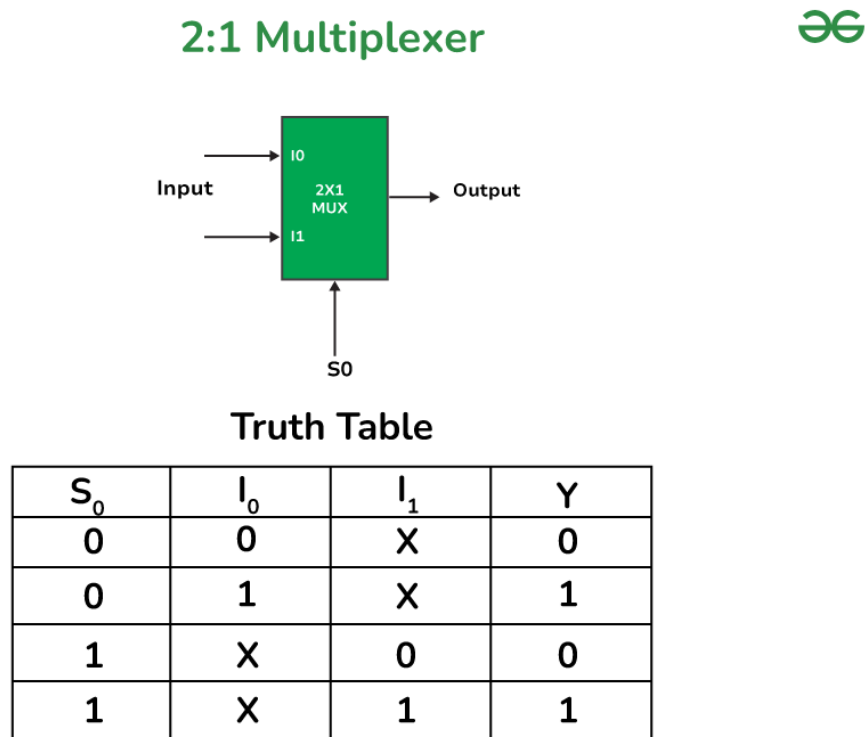
2x1 Multiplexer

The 2x1 is a fundamental circuit which is also known 2-to-1 multiplexer that are used to choose one [signal](#) from two inputs and transmits it to the output. The 2x1 mux has two input lines, one output line, and a single selection

line. It has various applications in digital systems such as in microprocessor it is used to select between two different data sources or between two different instructions.

Block Diagram of 2:1 Multiplexer with Truth Table

Given Below is the Block Diagram and Truth Table of 2:1 Mux. In this Block Diagram where I0 and I1 are the input lines, Y is the output line and S0 is a single select line.



Block Diagram of

2:1 Multiplexer with Truth Table

The output of the 2x1 Mux will depend on the selection line S0,

- When S is 0(low), the I0 is selected
- when S0 is 1(High), I1 is selected

Logical Expression of 2x1 Mux

Using the Truth Table ,the Logical Expression for Mux can be determined as

$$Y=S_0^{\sim}.I_0+S_0.I_1Y=S_0.I_0+S_0.I_1$$

Circuit Diagram of 2x1 Multiplexers

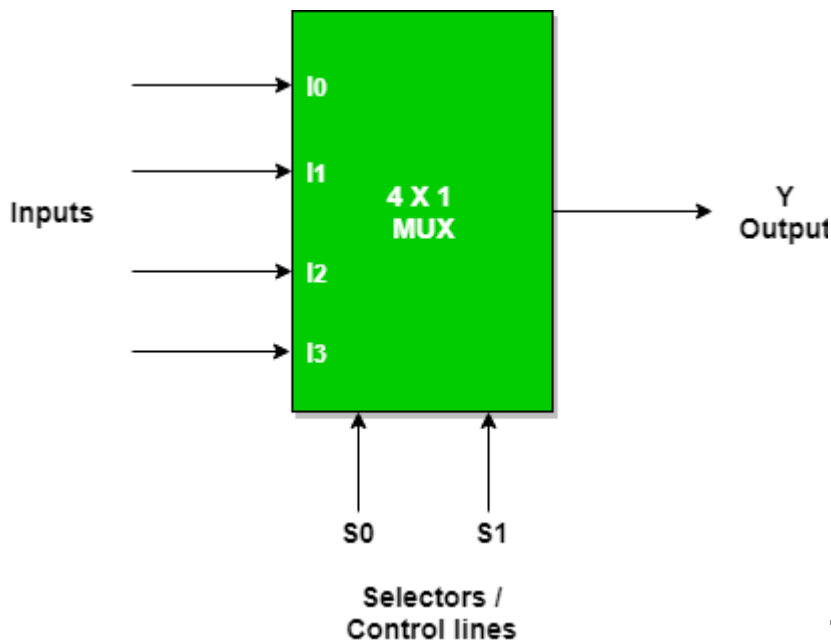
Using truth table the [circuit](#) diagram can be given as

4×1 Multiplexer

The 4x1 Multiplexer which is also known as the 4-to-1 multiplexer. It is a multiplexer that has 4 inputs and a single output. The Output is selected as one of the 4 inputs which is based on the selection inputs. The number of the Selection lines will depend on the number of the input which is determined by the equation $\log_2 n \log_2 n$, In 4x1 Mux the selection lines can be determined as $\log_2 4 = 2$, so two selections are needed.

Block Diagram of 4×1 Multiplexer

In the Given Block Diagram I0, I1, I2, and I3 are the 4 inputs and Y is the Single output which is based on Select lines S0 and S1.



the binary value of the selection lines

The output of the multiplexer is determined by

- When S1S0=00, the input I0 is selected.
- When S1S0=01, the input I1 is selected.
- When S1S0=10, the input I2 is selected.
- When S1S0=11, the input I3 is selected.

Truth Table of 4×1 Multiplexer

Given Below is the [Truth Table](#) of 4x1 Multiplexer

Truth Table

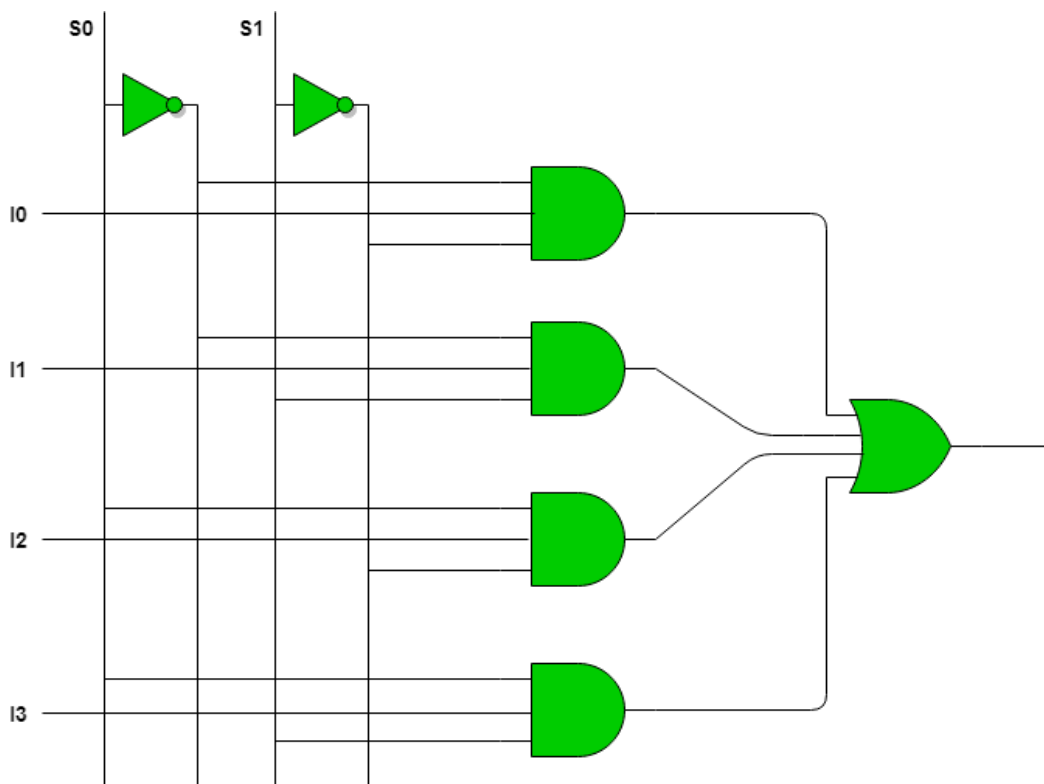
S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

So, final equation,

$$Y = S0'.S1'.I0 + S0'.S1.I1 + S0.S1'.I2 + S0.S1.I3$$

Circuit Diagram of 4x1 Multiplexers

Using truth table the circuit diagram can be given as



Multiplexer can act as universal combinational circuit. All the standard logic gates can be implemented with multiplexers.

Implementation of Different Gates with 2:1 Mux

Given below are the Implementation of Different gate using 2:1 Mux

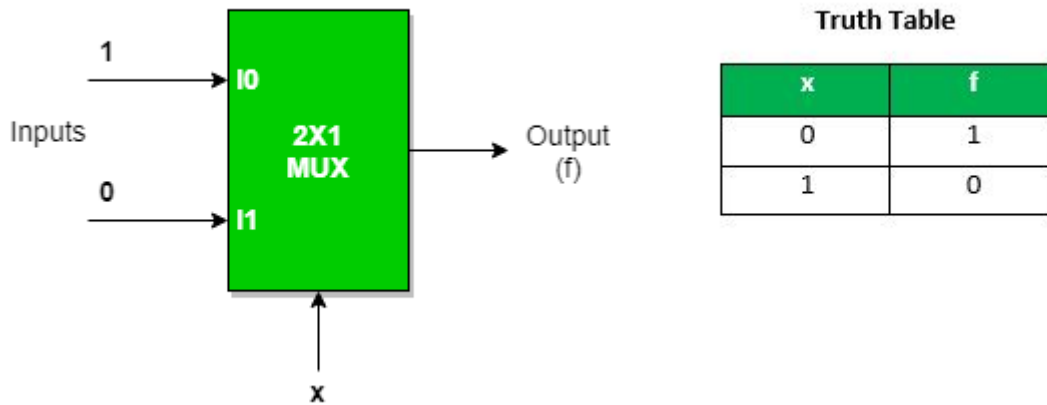
Implementation of NOT gate using 2 : 1 Mux

The Not gate from 2:1 Mux can be obtained by

- Connect the input signal to one of the data input lines(I0).

- Then connect a line (0 or 1) to the other data input line(I1)
- Connect the same input line Select line S0 which is connected to D0.

Given Below is the Diagram for the Logical Representation of NOT gate using 2 : 1 Mux

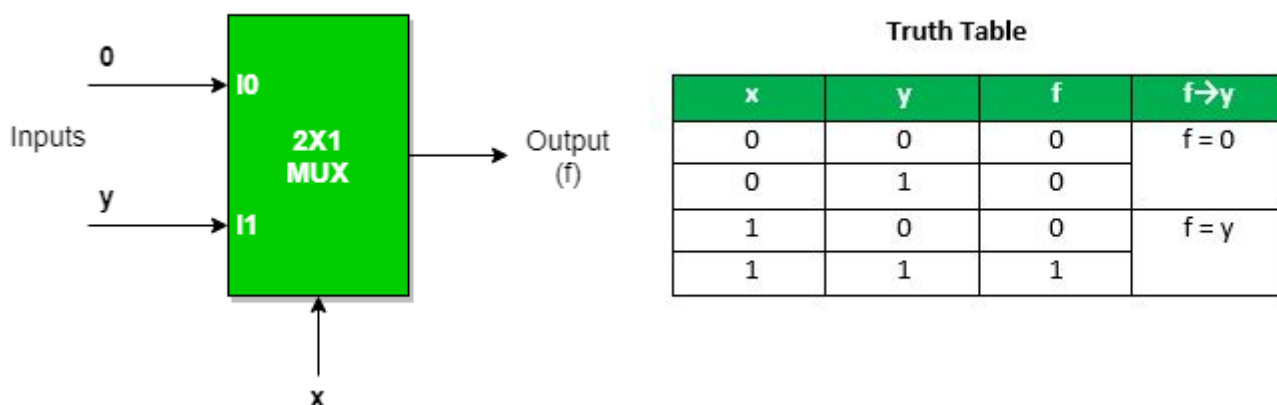


Implementation of AND gate using 2 : 1 Mux

The And gate from 2:1 Mux can be obtained by

- Connect the input Y to I1.
- Connect the input X to the selection line S0.
- Connect a line(0) to I0.

Given Below is the Diagram for the Logical Representation of **AND gate** using 2 : 1 Mux



For further more on the **Implementation of AND gate using 2 : 1 Mux**

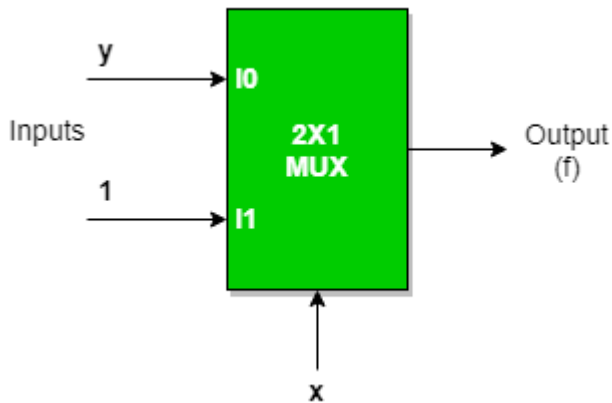
Implementation of OR gate using 2 : 1 Mux

The OR gate from 2:1 Mux can be obtained by

- Connect input X to the selection line S0.

- Connect input Y to I1.
- Connect Line(1) to I1.

Given Below is the Diagram for the Logical Representation of **OR gate_using 2 : 1 Mux**



Truth Table

x	y	f	f→y
0	0	0	f = y
0	1	1	
1	0	1	f = 1
1	1	1	

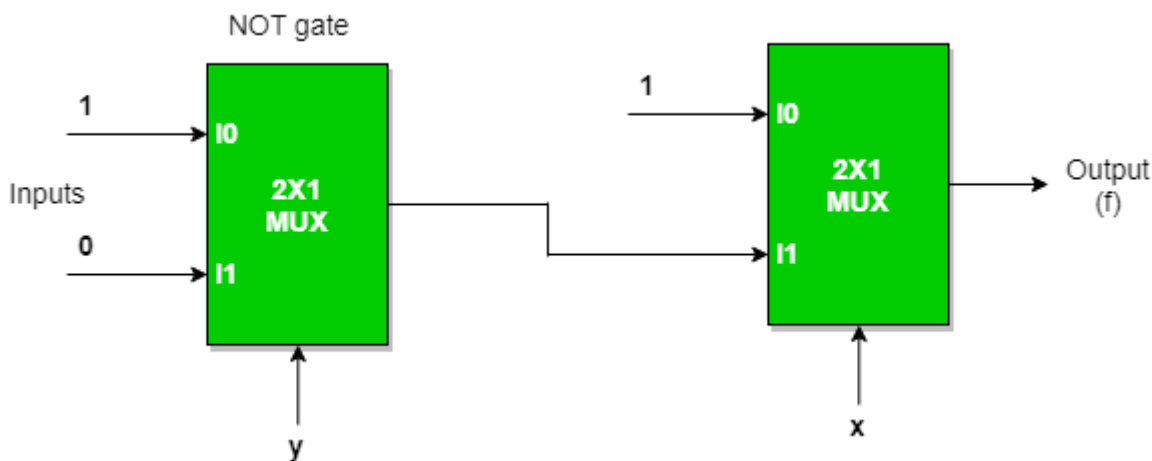
Implementation of NAND, NOR, XOR and XNOR gates requires two 2:1 Mux. First multiplexer will act as NOT gate which will provide complemented input to the second multiplexer.

Implementation of NAND gate using 2 : 1 Mux

The NAND gate from 2:1 Mux can be obtained by

- In first mux take inputs and 1 and 0 and y as selection line.
- In Second MUX the Output from mux is connected to I1.
- line(1) is given to the I0.
- x is given as selection line for the second Mux.

Given Below is the Diagram for the Logical Representation of **NAND gate_using 2 : 1 Mux**



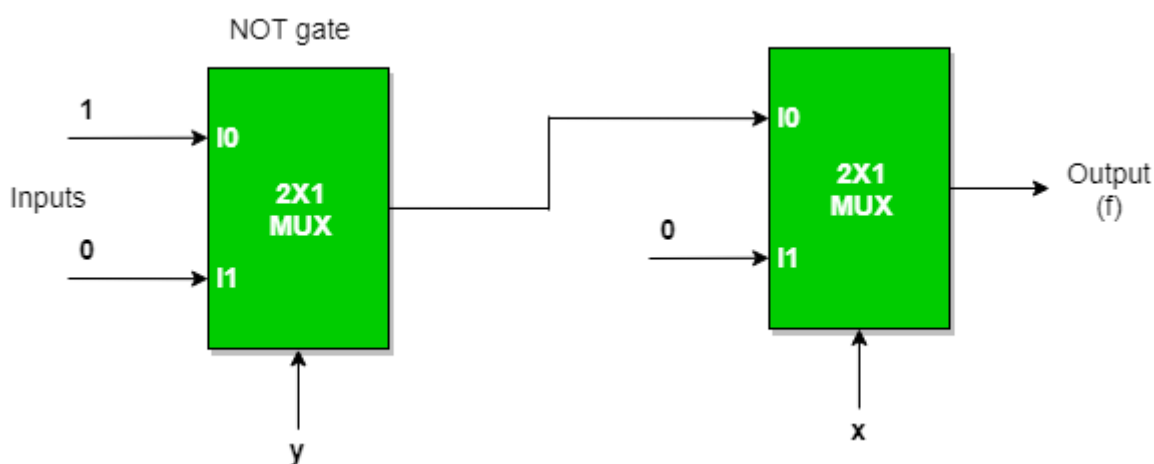
For further more on the **Implementation of NAND gate using 2 : 1 Mux**

Implementation of NOR gate using 2 : 1 Mux

The Nor gate from 2:1 Mux can be obtained by

- In first mux take inputs and 1 and 0 and y as selection line.
- In Second MUX the Output from mux is connected to I0.
- line(0) is given to the I1.
- x is given as selection line for the second Mux.

Given Below is the Diagram for the Logical Representation of **NOR gate_using 2 : 1 Mux**



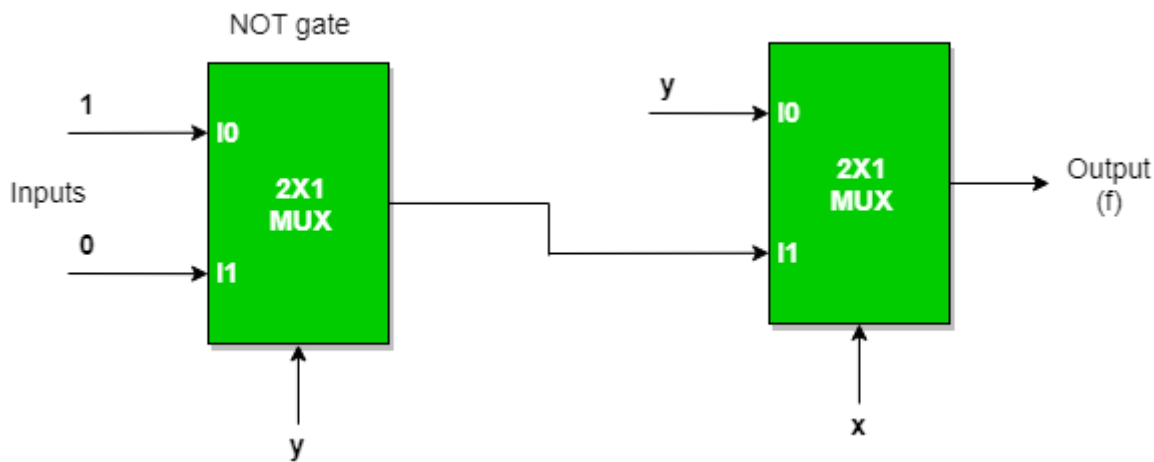
For further more on the **Implementation of NOR gate using 2 : 1 Mux**

Implementation of EX-OR gate using 2 : 1 Mux

The Nor gate from 2:1 Mux can be obtained by

- In first mux take inputs and 1 and 0 and y as selection line.
- In Second MUX the Output from mux is connected to I1.
- y is given to the I0.
- x is given as selection line for the second Mux.

Given Below is the Diagram for the Logical Representation of **EX-OR gate using 2 : 1 Mux**

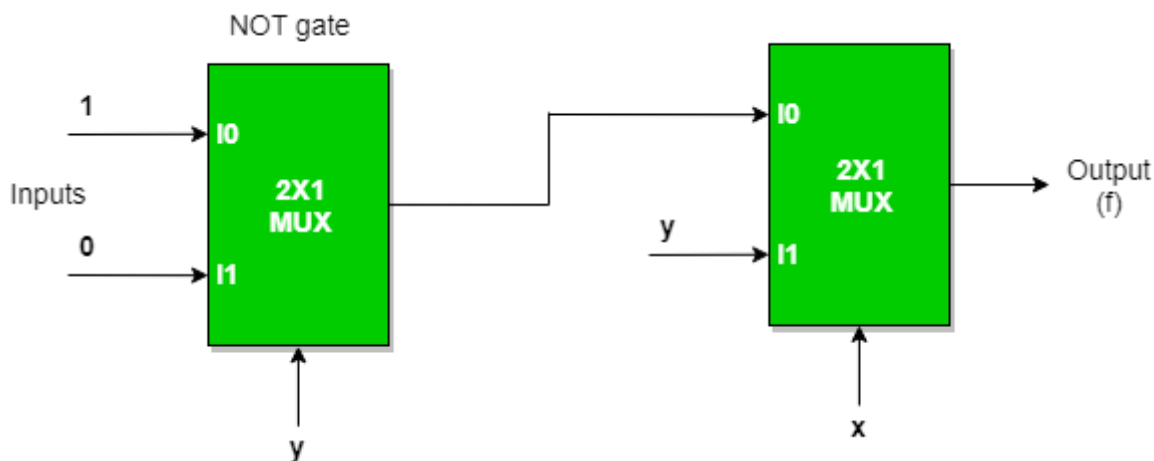


Implementation of EX-NOR gate using 2 : 1 Mux

Given Below is the Diagram for the Logical Representation of **EX-OR gate_using 2 : 1 Mux**

The Nor gate from 2:1 Mux can be obtained by

- In first mux take inputs and 1 and 0 and y as selection line.
- In Second MUX the Output from mux is connected to I0.
- y is given to the I1.
- x is given as selection line for the second Mux.



Implementation of Higher Order MUX using Lower Order MUX

Given Below are the Implementation of Higher Order MUX Using Lower Order MUX

4 : 1 MUX using 2 : 1 MUX

Three 2: 1 MUX are required to implement 4 : 1 MUX.

4 : 1 MUX using 2 : 1 MUX

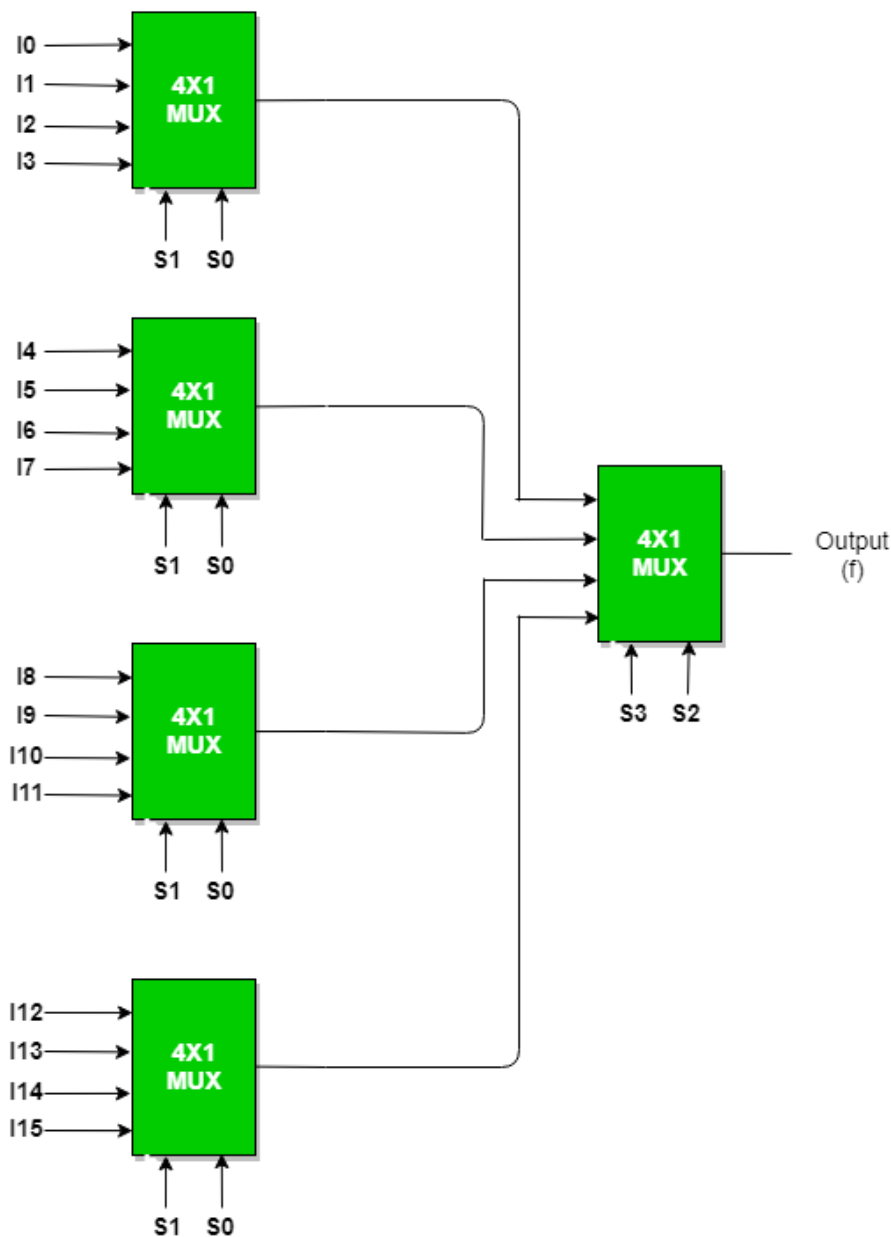
Similarly,

While an 8:1 MUX requires seven (7) 2:1 MUX, a 16:1 MUX requires fifteen (15) 2:1 MUX, and a 64:1 MUX requires sixty-three (63) 2:1 MUX. Hence, we can draw the conclusion that an $2^n:1$ MUX requires $(2^n - 1)$ 2:1 MUX. Hence, we can draw the conclusion that an $2^n:1$ MUX requires $(2^n - 1)$ 2:1 MUX.

16 : 1 MUX using 4 : 1 MUX

Given Below is the logical Diagram of 16:1 Mux Using 4:1 Mux

Inputs



In general, to implement $B : 1$ MUX using $A : 1$ MUX , one formula is used to implement the same.

$$B / A = K_1,$$

$$K_1 / A = K_2,$$

$$K_2 / A = K_3$$

$K_{N-1} / A = K_N = 1$ (till we obtain 1 count of MUX).

And then add all the numbers of MUXes = $K_1 + K_2 + K_3 + \dots + K_N$.

To implement 64 : 1 MUX using 4 : 1 MUX

Using the above formula, we can obtain the same.

$$64 / 4 = 16$$

$$16 / 4 = 4$$

$$4 / 4 = 1 \text{ (till we obtain 1 count of MUX)}$$

Hence, total number of 4 : 1 MUX are required to implement 64 : 1 MUX = $16 + 4 + 1 = 21$.

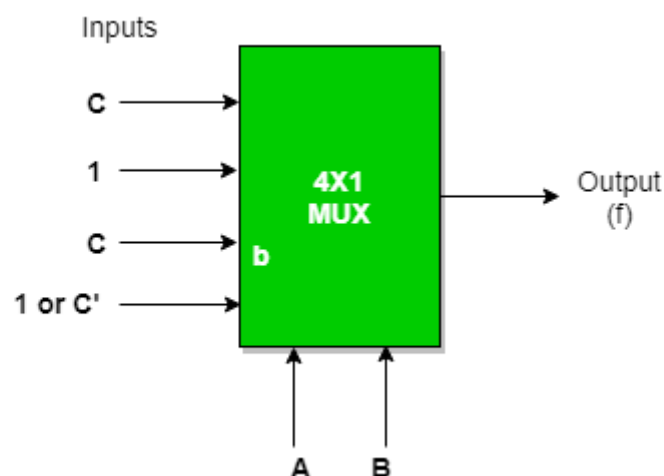
$$f(A, B, C) = \sum \sum (1, 2, 3, 5, 6) \text{ with don't care } (7)$$

Using A and B as the select lines for 4 : 1 MUX,

AB as select: Expanding the minterms to its boolean form and will see its 0 or 1 value in Cth place so that they can be placed in that manner.

A	B	C	f
0	0	0	C'
0	0	1	C
0	1	0	C'
0	1	1	C
1	0	0	C'
1	0	1	C
1	1	0	C'
1	1	1	C

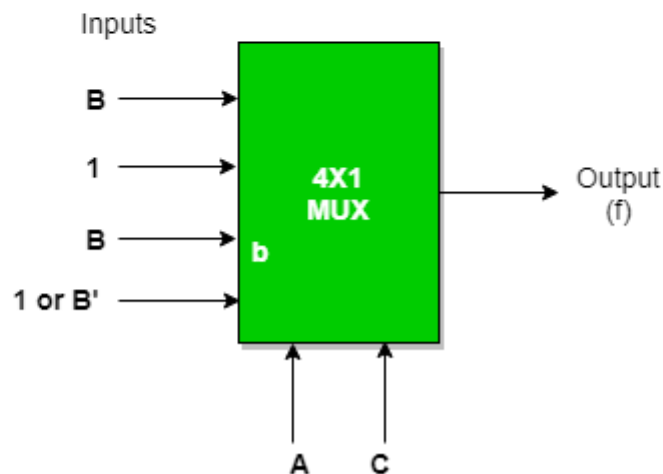
	I0	I1	I2	I3
C'	0	2	4	6
C	1	3	5	7 is don't care (can consider or not)
	C	1	C	1 (in case 7 is considered or C')



AC as select : Expanding the minterms to its [Boolean](#) form and will see its 0 or 1 value in Bth place so that they can be place in that manner.

A	B	C	f
0	0	0	B'
0	0	1	B'
0	1	0	B
0	1	1	B
1	0	0	B'
1	0	1	B'
1	1	0	B
1	1	1	B

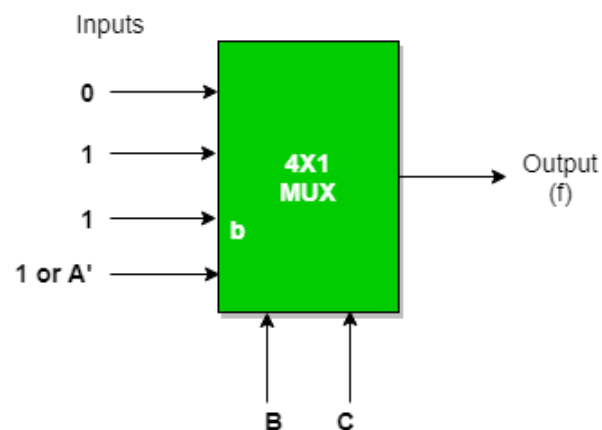
	I0	I1	I2	I3
B'	0	1	4	5
B	2	3	6	7 is don't care (can consider or not)
	B	1	B	1 (in case 7 is considered) or B'



BC as select: Expanding the [minterms](#) to its boolean form and will see its 0 or 1 value in Ath place so that they can be place in that manner.

A	B	C	f
0	0	0	A'
0	0	1	A'
0	1	0	A'
0	1	1	A'
1	0	0	A
1	0	1	A
1	1	0	A
1	1	1	A

	I0	I1	I2	I3
A'	0	1	2	3
A	4	5	6	7 is don't care (can considered or not)
	0	1	1	1 (in case 7 is considered) or A'



Applications of MUX

- **Data Routing** : The Mux is used for data routing in the digital system where they select one of the several data lines and re-route it the output.
- **Data Selection** : The Mux is used for data selection where they select data source according to the select lines.
- **Analog-to-Digital Conversion** : The Mux are used in [ADC](#) to select different analog input channels.
- **Address Decoding** : The Mux are used in [Microprocessors](#) or memory for address decoding.
- **Logic Function Implementation** : Mux can be used to implement various logic functions.

